PROJECT REPORT

TASK 19: FILTER DATASET RECORDS USING PANDAS

1. Abstract

This project demonstrates how to filter a dataset using Python and Pandas. A sample student performance dataset is used to apply numerical conditions, categorical conditions, and combinations of conditions.

The project illustrates Boolean indexing, AND and OR operations, and how filtering conditions can retrieve relevant records from a larger dataset.

2. Introduction

Data filtering is an important part of data analysis. Datasets often contain many records, but only a subset may be relevant to a particular question.

Pandas provides convenient ways to filter rows using Boolean conditions. These conditions can be combined to select records based on multiple requirements, such as marks, attendance, or department.

This project uses Google Colab to demonstrate these filtering techniques.

3. Objectives

The objectives are:

- To filter rows using numerical conditions.
- To filter records using categorical conditions.
- To combine multiple conditions using AND and OR.
- To display the resulting DataFrames.
- To count the matching records.
- To explain how each filtering condition works.

4. Tools and Technologies

Python: Used to implement the filtering program.

Pandas: Used to store the dataset in a DataFrame and filter records.

NumPy: Available for supporting numerical operations.

Google Colab: Used to execute the notebook interactively.

CSV: Used to export the sample dataset.

5. Dataset Description

The project uses a manually created student performance dataset containing 12 fictional records.

The dataset includes six columns:

Column	Description
student_id	Student identifier
name	Student name
department	Academic department
age	Student age
marks	Marks obtained
attendance	Attendance percentage

The data is used only to demonstrate filtering techniques.

6. Methodology

The project follows these steps:

1. Create the student dataset.
2. Convert it into a Pandas DataFrame.
3. Filter students based on marks.
4. Combine marks and attendance conditions using AND.
5. Combine alternative conditions using OR.
6. Filter records using department and marks.
7. Display each filtered DataFrame.
8. Count matching records and compare the results.
9. Export the dataset to CSV.

7. Filtering Operations

7.1 Filtering by Marks

The first example selects students whose marks are greater than or equal to 85.

The condition is:

`df["marks"] >= 85`

The result contains six matching students.

7.2 Multiple Conditions Using AND

The second example selects students whose marks are at least 80 and whose attendance is at least 85.

The condition is:

`(df["marks"] >= 80) & (df["attendance"] >= 85)`

Both conditions must be true for a record to appear in the result.

The result contains seven matching students.

7.3 Multiple Conditions Using OR

The third example selects students whose marks are at least 90 or whose attendance is at least 95.

The condition is:

`(df["marks"] >= 90) | (df["attendance"] >= 95)`

Either condition can be true for a record to be included.

The result contains four matching students.

7.4 Filtering by Department and Marks

The fourth example selects students belonging to the Computer Science department who scored at least 90.

This combines a categorical condition and a numerical condition using AND.

The result contains three students: Ananya, Sara, and Nisha.

8. Results

Filtering Condition	Number of Matching Records
Marks >= 85	6
Marks >= 80 AND attendance >= 85	7
Marks >= 90 OR attendance >= 95	4
Computer Science AND marks >= 90	3

These results demonstrate that different conditions retrieve different subsets of the original dataset.

9. Boolean Indexing

Boolean indexing uses a condition to determine which rows should be included in a filtered DataFrame.

In Pandas, comparison expressions produce Boolean values. The resulting Boolean series can be used inside square brackets to return matching rows.

When combining multiple conditions, the `&` operator represents AND and the `|` operator represents OR. Each condition should be enclosed in parentheses to ensure the expressions are evaluated correctly.

10. Applications

Dataset filtering is useful in:

- Student performance analysis.
- Sales and revenue analysis.
- Employee data analysis.
- Customer segmentation.
- Exploratory data analysis.
- Preparing training data for machine-learning models.

11. Limitations

The project uses a small, manually created dataset. The examples demonstrate basic filtering and do not include advanced data cleaning or predictive modelling.

12. Future Enhancements

The project could be extended to filter larger datasets, accept user-defined conditions, handle missing data, generate visualizations, and export filtered records into separate CSV files.

13. Conclusion

This project demonstrates how Pandas can filter records using numerical, categorical, and combined conditions. It provides practical experience with Boolean indexing, logical operators, DataFrames, and record counts.

These concepts are useful for exploratory data analysis and for preparing relevant subsets of data for machine-learning workflows.

14. References

1. Pandas documentation: https://pandas.pydata.org/docs/
2. Pandas indexing guide: https://pandas.pydata.org/docs/user_guide/indexing.html

15. Project Links

GitHub Repository: https://github.com/venky1318/Filter_Dataset_Records